

## Criteria-for-Modularization

O artigo "On the Criteria To Be Used in Decomposing Systems into Modules", escrito por David L. Parnas em 1972, continua sendo uma leitura essencial porque ataca a raiz de muitos problemas que enfrentamos até hoje em arquitetura de software e integração de sistemas. A tese central de Parnas é que a forma como dividimos um sistema em módulos não deve ser apenas uma questão de organização de código, mas sim uma estratégia para aumentar a flexibilidade e diminuir o tempo de desenvolvimento. Ele argumenta que a eficácia dessa divisão depende inteiramente dos critérios que usamos, e é aqui que ele introduz o conceito revolucionário de "ocultação de informação" (*information hiding*).

Para provar seu ponto, Parnas utiliza o exemplo de um sistema de indexação KWIC (*Key Word In Context*), comparando duas abordagens de design. Na primeira, que ele chama de convencional, a divisão segue o fluxo lógico do processamento, um módulo para entrada de dados, um para rotacionar as linhas, um para ordenar alfabeticamente e outro para exibir o resultado. Embora pareça intuitiva, essa abordagem é baseada em um fluxograma de execução. O grande problema aqui é que todos os módulos acabam compartilhando o conhecimento detalhado sobre como os dados são representados fisicamente na memória. Se você decidir mudar a forma como os caracteres são armazenados ou como os ponteiros funcionam, terá que alterar praticamente todos os módulos do sistema. O sistema torna-se rígido, e qualquer mudança gera um efeito cascata que inviabiliza o desenvolvimento independente e a manutenção a longo prazo.

Em contrapartida, a segunda modularização proposta por Parnas foca em esconder decisões de design difíceis ou que tenham alta probabilidade de mudar. Em vez de módulos que representam "passos", ele propõe módulos que gerenciam responsabilidades específicas. Por exemplo, o módulo de "Armazenamento de Linhas" expõe apenas funções abstratas para acessar um caractere ou contar palavras, ocultando completamente se os dados estão em memória, em disco ou compactados. Essa abstração cria uma barreira de proteção, se o

formato de armazenamento mudar, apenas esse módulo precisa ser reescrito. Os outros módulos continuam chamando as mesmas funções sem nem perceber que o "motor" interno mudou. Isso permite que equipes diferentes trabalhem de forma muito mais independente, já que os contratos (interfaces) são simples e baseados em funções, e não em estruturas de dados complexas e compartilhadas.

Além da facilidade de mudança, essa abordagem melhora drasticamente a compreensibilidade do sistema. Parnas observa que, no modelo convencional, é impossível entender como o módulo de saída funciona sem antes entender as peculiaridades de todos os processos anteriores que mexeram nos dados. Com a ocultação de informação, o sistema torna-se compreensível em partes, você pode estudar a lógica de um módulo isoladamente, confiando apenas na sua interface definida. Ele também aborda a questão da eficiência, reconhecendo que muitas chamadas de função podem custar caro em termos de performance. No entanto, sua solução é técnica, devemos manter a modularização clara para o desenvolvedor, mas utilizar ferramentas de compilação ou montagem que otimizem o código final para execução, separando a lógica da organização da eficiência da máquina.

Por fim, o autor diferencia uma boa modularização de uma estrutura hierárquica. Uma hierarquia baseada na relação de "dependência" permite que partes do sistema sejam reutilizadas de forma independente. Um módulo de armazenamento de dados bem desenhado pode ser a base para um sistema de indexação hoje e para um banco de dados amanhã, desde que ele não dependa dos níveis superiores para funcionar. A conclusão de Parnas é direta, para construir sistemas resilientes, devemos abandonar a ideia de que módulos são apenas passos de um fluxograma e passar a vê-los como cápsulas que protegem o restante do sistema contra decisões de design instáveis.